

Report 1

Andrei Petrea

Faculty of Automatic Control and Computers

National University of Science and Technology POLITEHNICA Bucharest

June 13, 2026

Abstract

This report examines container image minimization in the context of modern day use-cases. It provides an overview of the state-of-the-art tools and techniques for minimizing container images, as well as a detailed description of my work in developing a new tool for this purpose. The report concludes with a comparison of my tool with existing solutions and a discussion of potential future work in this area.

1 Introduction

It has been almost a decade since the famous *Attention Is All You Need* [4] paper was published, which began the era of artificial intelligence (AI) development, culminating in the creation of large language models (LLMs) such as ChatGPT.

In the past few years, LLMs have gradually transitioned from simple chatbots into highly sophisticated tools that can perform a wide range of tasks. This transition is characterized by a migration away from simple prompt-and-response dynamics toward continuous, goal-oriented autonomy. Modern agentic frameworks leverage the underlying model not just for linguistic fluency, but as a dynamic engine for systemic interaction—capable of maintaining context over extended horizons, utilizing specialized tools, and executing complex, multi-stage workflows within broader software architectures.

As such, it is natural to explore how LLMs can be utilized to build minimal container images. In this context, I created an agentic workflow where LLMs are leveraged to fuzz the application and in order to find as many execution paths in order to more accurately determine the minimal set of dependencies required to run and interact with the application.

2 Architecture

What are AI agents?

An AI agent is an autonomous system that performs tasks by designing and executing workflows with available tools. They overcome the limitations of traditional LLMs by being augmented with memory, tool use, and the ability to plan and execute multi-step processes. This allows them to handle complex tasks that require reasoning, decision-making, and interaction with external systems [2].

The typical AI agent flow consists of the following three components:

- **Goal initialization & planning:** Human-defined goals and rules guide the agent to decompose tasks into subtasks and create a plan.
- **Reasoning with tools:** The agent calls external datasets, APIs, or other agents to fill information gaps, then updates its knowledge and refines its plan.
- **Learning & reflection:** Feedback from humans, other agents, or HITL(Human-in-the-Loop) interactions is stored in memory, enabling iterative improvement and personalization.

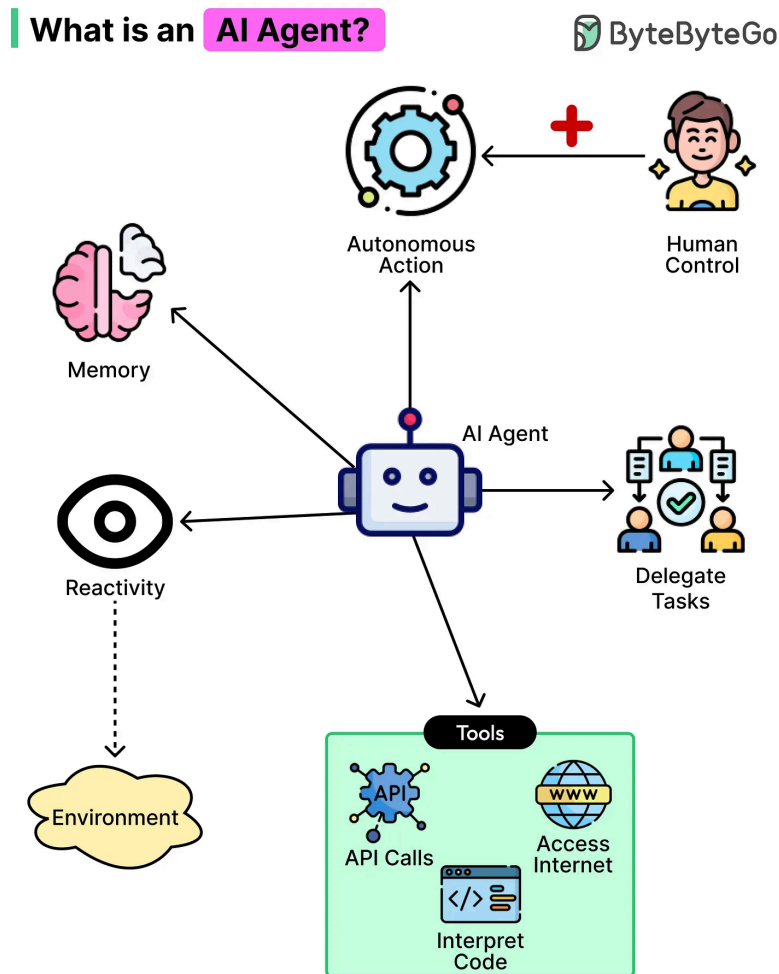


Figure 1: AI agents architecture

Using agents to minimize applications

Owing to their above mentioned capabilities, I identified two ways they can be used in my project:

- **AI Fuzzing Agent:** An agent that fuzzes the application to trigger state changes in order to identify hidden dependencies
- **AI Testing Agent:** An agent that tests the minimize application to ensure that it runs the same as the original application and that it is free of bugs

For this report, I will focus on the *AI Fuzzing Agent*, leaving the testing one for future work.

As you can recall from the previous report [3], my application tries to build the minimal container image in three steps: a static analysis using ldd, a dynamic analysis using strace and a brute-force binary search in case the first two failed. In theory, the final binary stage is sufficient since, given enough time, it will eventually find the minimal set. However, in practice, it is very resource-intensive and the final result is not much smaller than the original one (multiple binary-searches one after the other will asymptotically approach the minimal set).

As such, the best way to improve the performance of the project is to improve the dynamic analysis stage, which is where the AI Fuzzing Agent comes in. As the name implies, the agent will act as a fuzzer, an automated testing technique that injects input into a program to trigger state changes [1].

MUTATION FUZZING WORKFLOW

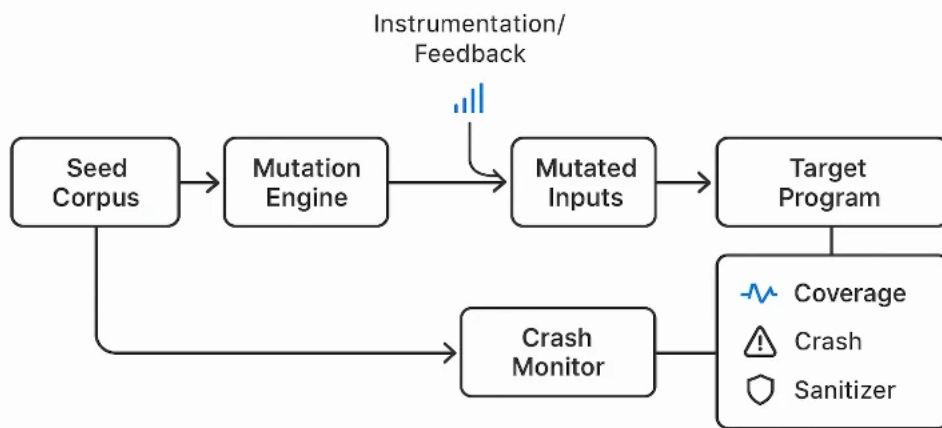


Figure 2: A typical fuzzing flow

The goal of the agent is to send input to the application whilst it's being traced by strace, in order to trigger as many state changes to uncover any hidden dependencies that the app may need in order to run.

3 My Work

Considering the disadvantages of existing tools for minimizing container images, I developed a new tool called *dockermimizer* that aims to address these issues.

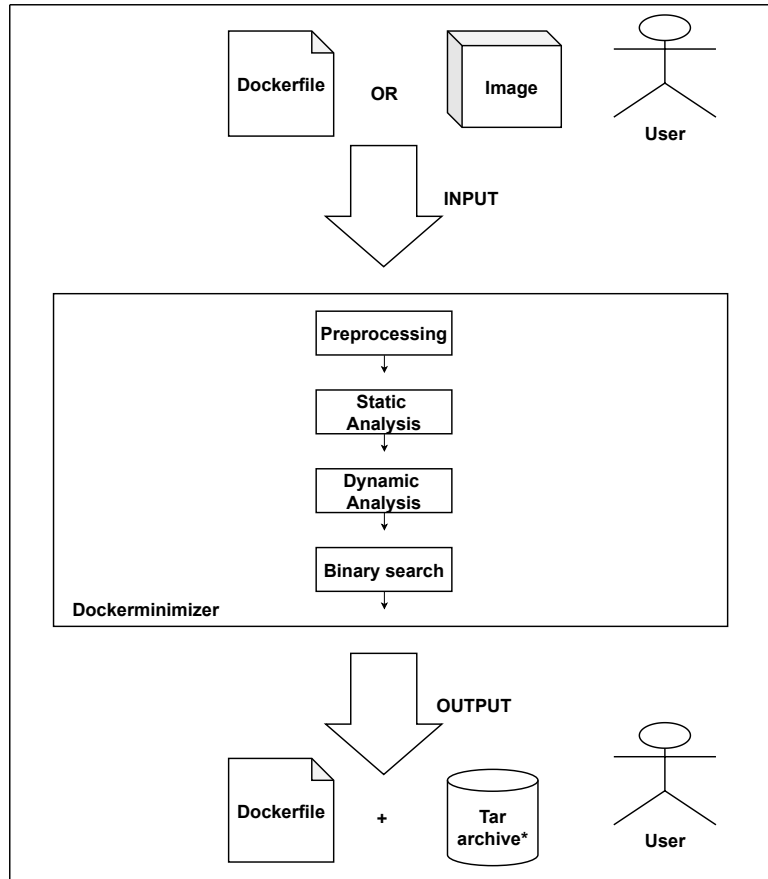


Figure 3: High-level architecture of the application

My tool is implemented in *Go* and is designed to be modular and extensible. It takes as input either a Dockerfile or an existing Docker image and performs static, dynamic analysis and binary search to minimize the application containerized, outputting the Dockerfile and if needed the tarball of all dependency files. It takes the following command-line arguments:

- **-file**: The path to the Dockerfile to be minimized.
- **-image**: The name of an images from a Docker registry to be minimized
- **-max_limit**: How many times should the binary search procedure be run for
- **-debug**: Enable logging of actions
- **-timeout**: How long should the minimal Docker container run before being declared as successful
- **-strace_path**: The path to a statically-linked strace binary

- **–binary_search:** Decide whether to continue the minimization process with a binary search if dynamic analysis failed

The tool expands from Slim by firstly performing static analysis to conserve processing resources and has a brute-force binary search approach in order to ensure that the minimization process terminates successfully even in cases where dynamic analysis fails.

Binary Search

Since this is a novel approach, I will briefly describe how the binary search procedure works.

The binary search procedure is a fail-safe mechanism that is used when the dynamic analysis fails to yield any results. It works by splitting the original image's filesystem into two halves and checking if the application can run successfully with only one of the halves. If it can, then the other half is split into two again and added to the first half. Once the application successfully runs, the files that remained in the second pile are discarded and the new files become the new baseline. This process is repeated until the maximum number of iterations is reached or the image cannot be minimized any further.

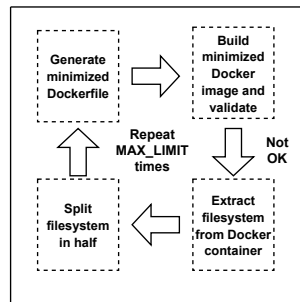


Figure 4: Binary search procedure for minimizing the application

4 Conclusion and Further Work

My tool is still a work in progress and there are several features that are missing or need improvement. I have created a comparison table between my tool and Slim in order to highlight the differences and similarities between the two tools.

Table 1: Comparison between Dockerminimizer and Slim

Feature	Dockerminimizer	Slim
Implementation Language	Go	Go
Analysis Technique	Static Analysis ('ldd') Dynamic Analysis ('strace') Binary Search (Brute Force)	Dynamic Analysis (Sensors/Probes) Static Metadata Analysis
Output Format	Minimized Dockerfile + Filesystem Tarball	Minified Docker Image ('.slim') (Opaque Binary)
Fallback Mechanism	Binary Search (Automated recovery of missing files)	Manual Configuration (User must manually include paths)
Base Image	FROM scratch	Custom Minified Distro
Transparency	High (User can inspect/edit the resulting Dockerfile)	Low (Black-box image generation)
Additional Features	None	Full Suite (xray, lint, debug, merge, registry, vulnerability)

The main advantage of my tool is the transparency of the resulting Dockerfile, which allows users to inspect and modify the minimized image as needed and the automated binary search procedure that ensures that the minimization process terminates successfully even in cases where dynamic analysis fails. However, in order to ensure the viability of the tool, I need to either implement some of the additional features that Slim offers or improve on the minimization process in order to achieve better results than Slim.

I have also identified several areas for further work, including:

- **Dynamic Analysis Improvements:** In addition to *strace* (which uses *ptrace*), I could explore other dynamic analysis techniques such as eBPF or systemtap to capture more comprehensive runtime information and potentially improve the accuracy of the minimization process.
- **Binary Search Optimization:** The current brute-force binary search approach can be time-consuming, especially for larger images. I could investigate more efficient search algorithms or heuristics to reduce the number of iterations needed to find the minimal set of files.
- **Security Analysis Integration:** Integrating security analysis tools to identify and remove vulnerable dependencies could enhance the security posture of the minimized images.
- **User Interface Enhancements:** Developing a more user-friendly interface, such as a web dashboard or CLI improvements, could make the tool more accessible to a wider range of users.

- **Support for Additional Container Formats:** Extending support to other container formats (e.g., OCI images) could broaden the applicability of the tool in different container ecosystems.
- **Application Path Analysis:** Implementing a more sophisticated application path analysis to better understand the dependencies and interactions between files could lead to more effective minimization.

References

- [1] BlackDuck. Fuzz testing. <https://www.blackduck.com/glossary/what-is-fuzz-testing.html>.
- [2] A. Gutowska. Ai agents defined. <https://www.ibm.com/think/topics/ai-agents>.
- [3] A. Petrea. Report 1, 2026.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. <https://arxiv.org/abs/1706.03762>, 2017.